

CSC 326 – Compiler Construction

Overview

Credits

- ❑ Slides have been adapted from slides by Hal Perkins, University of Washington
- ❑ <https://courses.cs.washington.edu/courses/csep501/08wi/>

And the point is...

❓ Execute this!

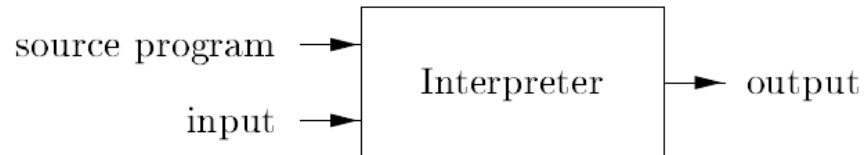
```
int nPos = 0;
int k = 0;
while (k < length) {
    if (a[k] > 0) {
        nPos++;
    }
}
```

❓ How?

Interpreters & Compilers

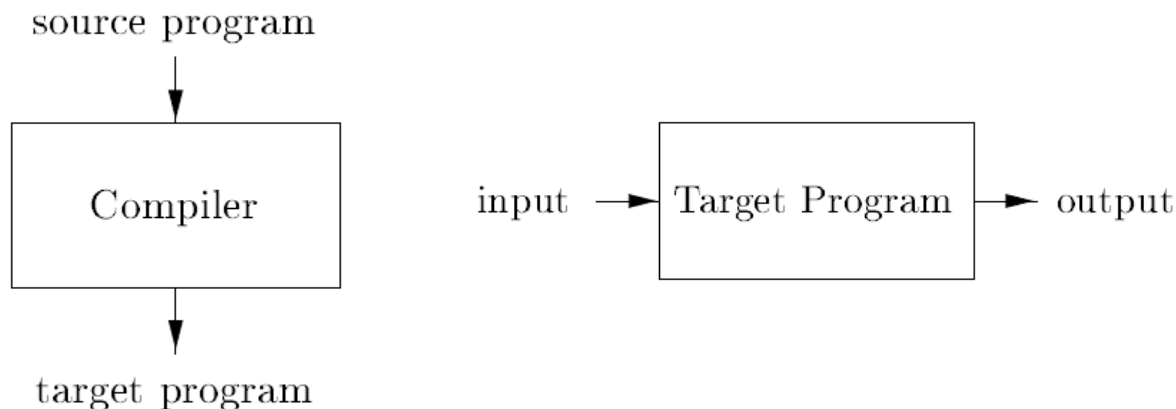
? Interpreter

- ? A program that reads a source program and produces the results of executing that program



? Compiler

- ? A program that translates a program from one language (the *source*) to another (the *target*)



Common Issues

- ❑ Compilers and interpreters both must read the input – a stream of characters – and “understand” it; *analysis*

```
w h i l e ( k < l e n g t h ) { <nl> <tab> i f ( a [ k ] > 0  
) <nl> <tab> <tab> { n P o s + + ; } <nl> <tab> }
```

Interpreter

❑ Interpreter

- ❑ Execution engine

- ❑ Program execution interleaved with analysis

```
running = true;  
while (running) {  
    analyze next statement;  
    execute that statement;  
}
```

- ❑ Usually need repeated analysis of statements (particularly in loops, functions)

- ❑ But: immediate execution, good debugging & interaction

Compiler

- ❑ Read and analyze entire program
- ❑ Translate to semantically equivalent program in another language
 - ❑ Presumably easier to execute or more efficient
 - ❑ Should “improve” the program in some fashion
- ❑ Offline process
 - ❑ Tradeoff: compile time overhead (preprocessing step) vs execution performance

Typical Implementations

❑ Compilers

- ❑ FORTRAN, C, C++, Java, COBOL, etc. etc.
- ❑ Strong need for optimization in many cases

❑ Interpreters

- ❑ PERL, Python, Ruby, awk, sed, shells, Scheme/Lisp/ML, postscript/pdf, Java VM
- ❑ Particularly effective if interpreter overhead is low relative to execution cost of individual statements

Hybrid approaches

❑ Well-known example: Java

- ❑ Compile Java source to byte codes – Java Virtual Machine language (.class files)

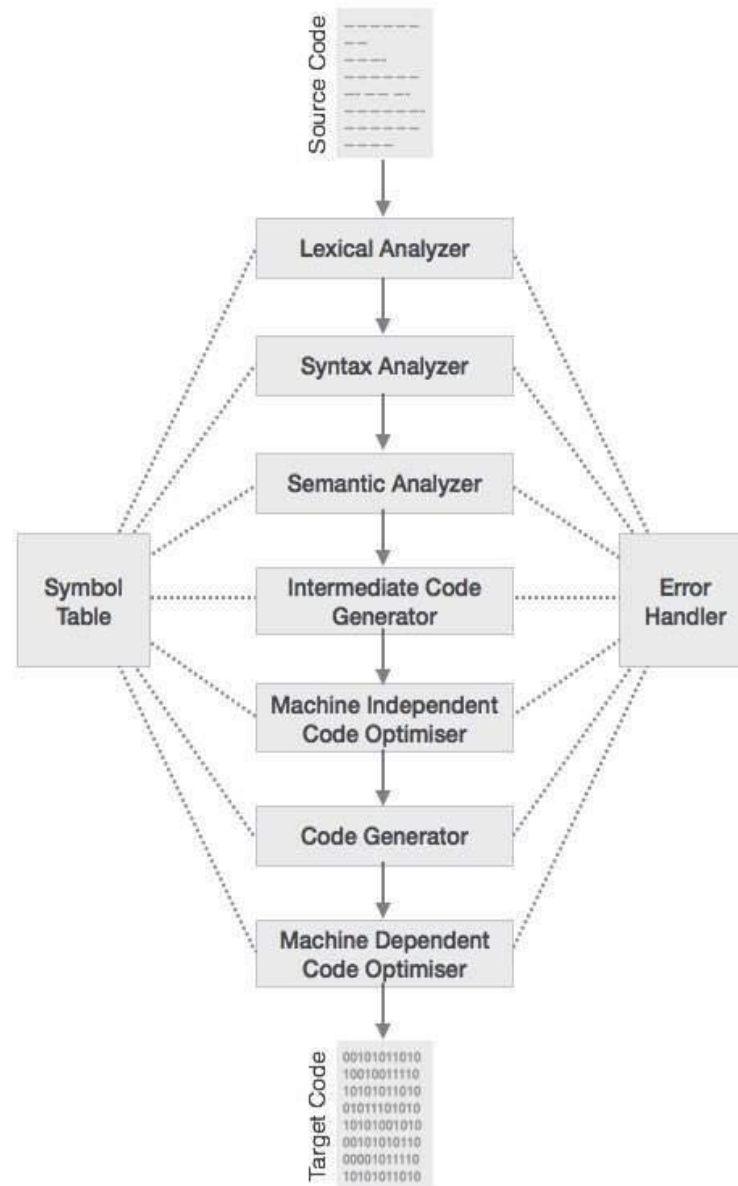
❑ Execution

- ❑ Interpret byte codes directly, or

- ❑ Compile some or all byte codes to native code

- Just-In-Time compiler (JIT) – detect hot spots & compile on the fly to native code – standard these days

Phases of a Compiler



Lexical Analyzer/Scanner

- ❑ Reads characters from the source program.
- ❑ Groups characters into **lexemes**
 - ❑ Meaningful sequences of characters that "go together"
 - ❑ Removes white space, comments
- ❑ Lexemes corresponds to a **tokens**
 - ❑ scanner returns next token (plus maybe some additional information) to the parser
 - ❑ Token : <token_name, attribute_value>
 - ❑ token_name is used by the parser
 - ❑ attribute_value points to the entry of the symbol table for this token
- ❑ Scanner also discovers lexical errors (e.g., erroneous characters such as # in java).

Example: Lexemes and Tokens

Lexeme:	;	=	index	tmp	21	64.32
Token:	SEMI-COLON	ASSIGN	ID	ID	NUM_INT	NUM-FLOAT

Example:

position = initial + rate * 60

↓
Lexical Analyzer

↓
<id, 1> <=> <id, 2> <+> <id, 3> <*> <60>

1	position	...
2	initial	...
3	rate	...

SYMBOL TABLE



Why is Lexical Analysis Needed?

- ❑ Simplifies parsing
- ❑ Makes compiler faster
- ❑ Separates concerns: tokenizing vs syntax analysis
- ❑ Detects invalid characters early
- ❑ Supports regular expressions for token patterns



Syntax Analyzer/Parser

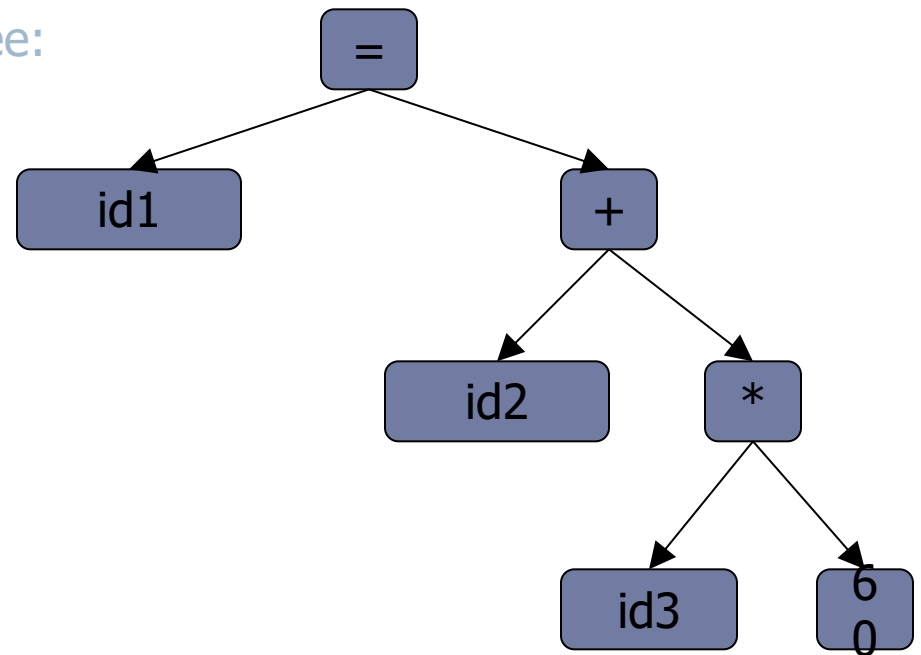
- ❑ Groups tokens into "grammatical phrases/sentences", discovering the underlying structure of the source program.
- ❑ Finds syntax errors.
For example, in Java the source code
`position = 5 ; *`
corresponds to the sequence of tokens:
`IDENT ASSIGN_OP INT SEMI-COLON MULT_OP`
All are legal tokens, but that sequence of tokens is erroneous.
- ❑ Might find some "static semantic" errors
 - ❑ a use of an undeclared variable or variables that are multiply declared
- ❑ Might generate code, or build some intermediate representation of the program such as a parse tree / abstract-syntax tree (AST)

Example Parse tree/Abstract Syntax Tree

Sequence of tokens:

`<id1, 1> <=> <id2,2> <+> <id3,3> <*> <60>`

Abstract-Syntax Tree:



- Interior nodes are **operators**.
- A node's children are **operands**.



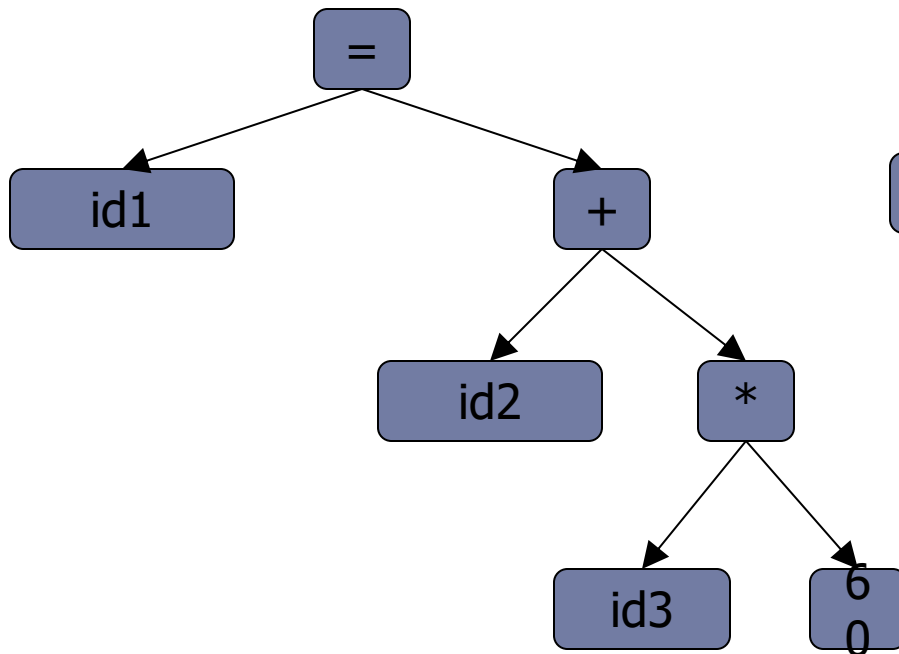
Semantic Analyzer

- ❑ Uses the Syntax tree
- ❑ Checks for additional static semantic consistency of source code
 - ❑ Performs type checking
 - ❑ may perform automatic type conversion/coercion
 - ❑ Using undeclared variables
 - ❑ Function called with improper arguments
 - ❑ Number and type of arguments
 - ❑ Array variables used without array syntax
 - ❑ Left hand side of an assignment must be a variable (sometimes called an L-value)
- ❑ Annotates and/or changes the AST

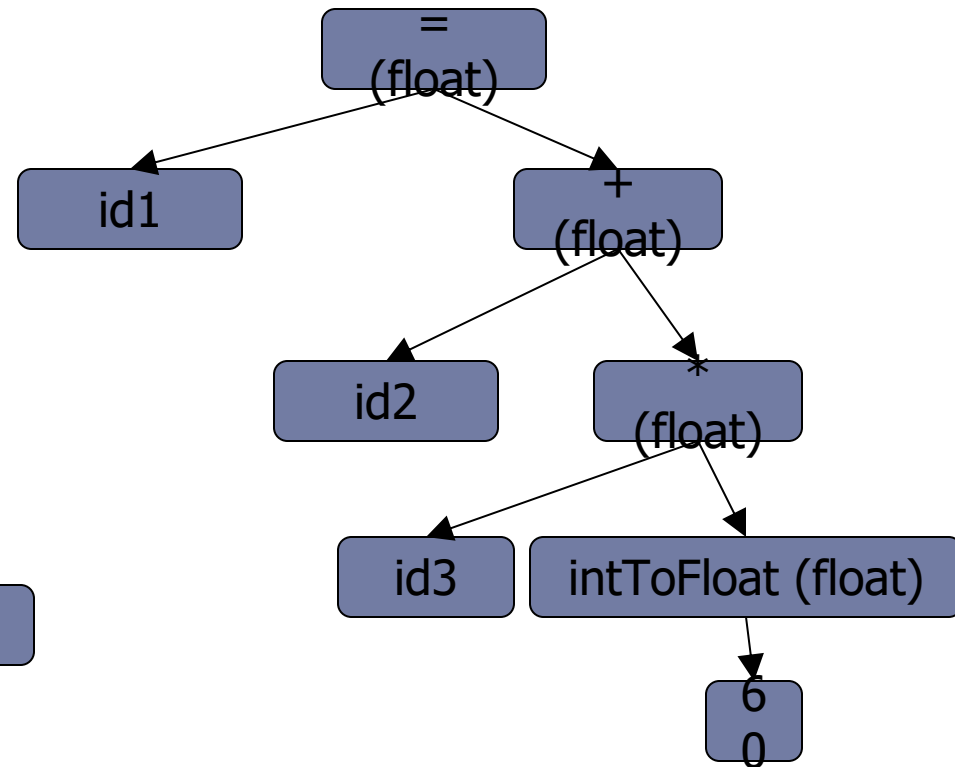


Example Semantic Analysis

Abstract-Syntax Tree:



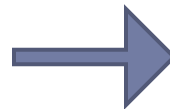
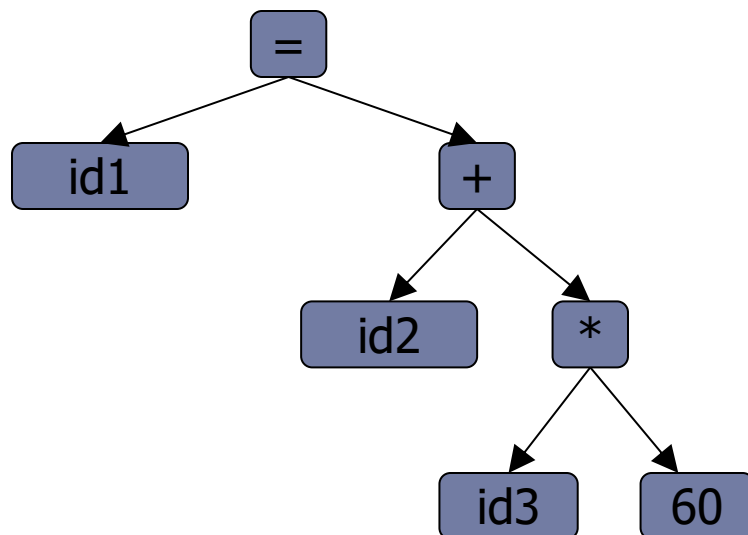
Annotated Abstract-Syntax Tree:



Intermediate Code Generator

- ❑ Translates from abstract-syntax tree to low-level machine like intermediate code
 - ❑ Should be easy to produce
 - ❑ Should be easy to translate to machine code
- ❑ One possibility is 3-address code
 - ❑ each instruction involves at most 3 operands

❑ Example:



```
temp1 = inttofloat(60)
temp2 = id3 * temp1
temp3 = id2 + temp2
id1 = temp3
```

Why Do We Need Intermediate Code?

- **Portability:**
Same front-end → can target multiple machines.
- **Optimization:**
Many optimizations (constant folding, dead code removal) are done here.
- **Simplified code generation:**
Converts complex high-level constructs into simple low-level steps.
- **Decouples front-end and back-end:**
Front-end: language dependent
Back-end: hardware dependent
ICG sits in the middle.

Example

$x = (a + b) * (c - d);$

Intermediate Code

$t1 = a + b$

$t2 = c - d$

$t3 = t1 * t2$

$x = t3$

Optimizer

❑ Tries to improve code to

❑ Run faster

❑ Be shorter

❑ Consume less energy

❑ Example

```
temp1 = inttofloat(60)
temp2 = id3 * temp1
temp3 = id2 + temp2
id1 = temp3
```



```
temp1 = id3 * inttofloat(60)
id1 = id2 + temp1
```

Code Generator

❑ Generate object code from (optimized) intermediate code

- ❑ Instruction selection & scheduling
- ❑ Register allocation

❑ Example

```
temp1 = id3 * inttofloat(60)
id1 = id2 + temp1
```

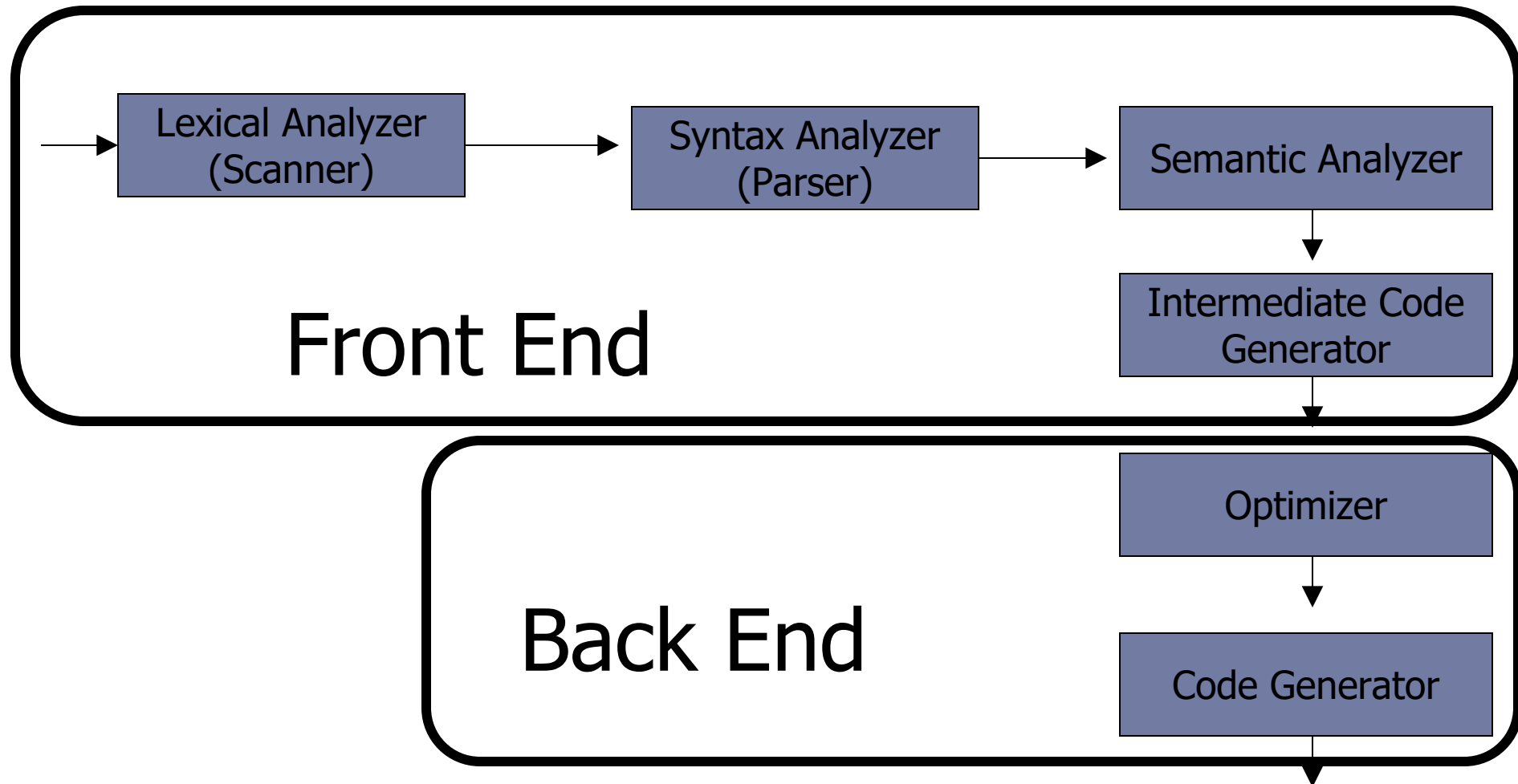


```
LDF R2, id3
MULF R2, R2, #60.0
LDF R1, id2
ADDF R1, R1, R2
STF id1, R1
```

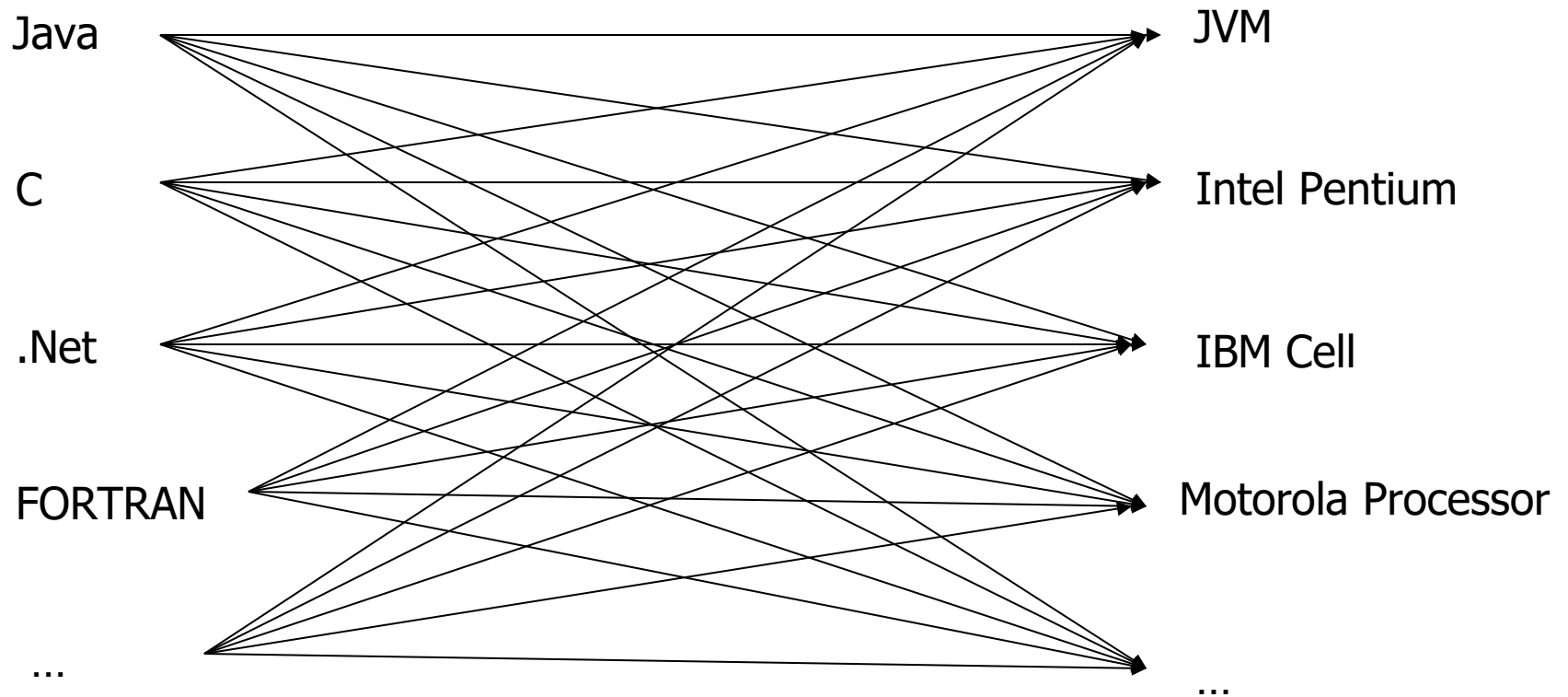
Symbol Tables

- ❑ Keep track of names declared in the program
- ❑ Separate level for each scope
- ❑ Used to analyze static semantics:
 - ❑ Variables should not be declared more than once in a scope
 - ❑ Variables should not be used before being declared
 - ❑ Parameter types for methods should match method declaration

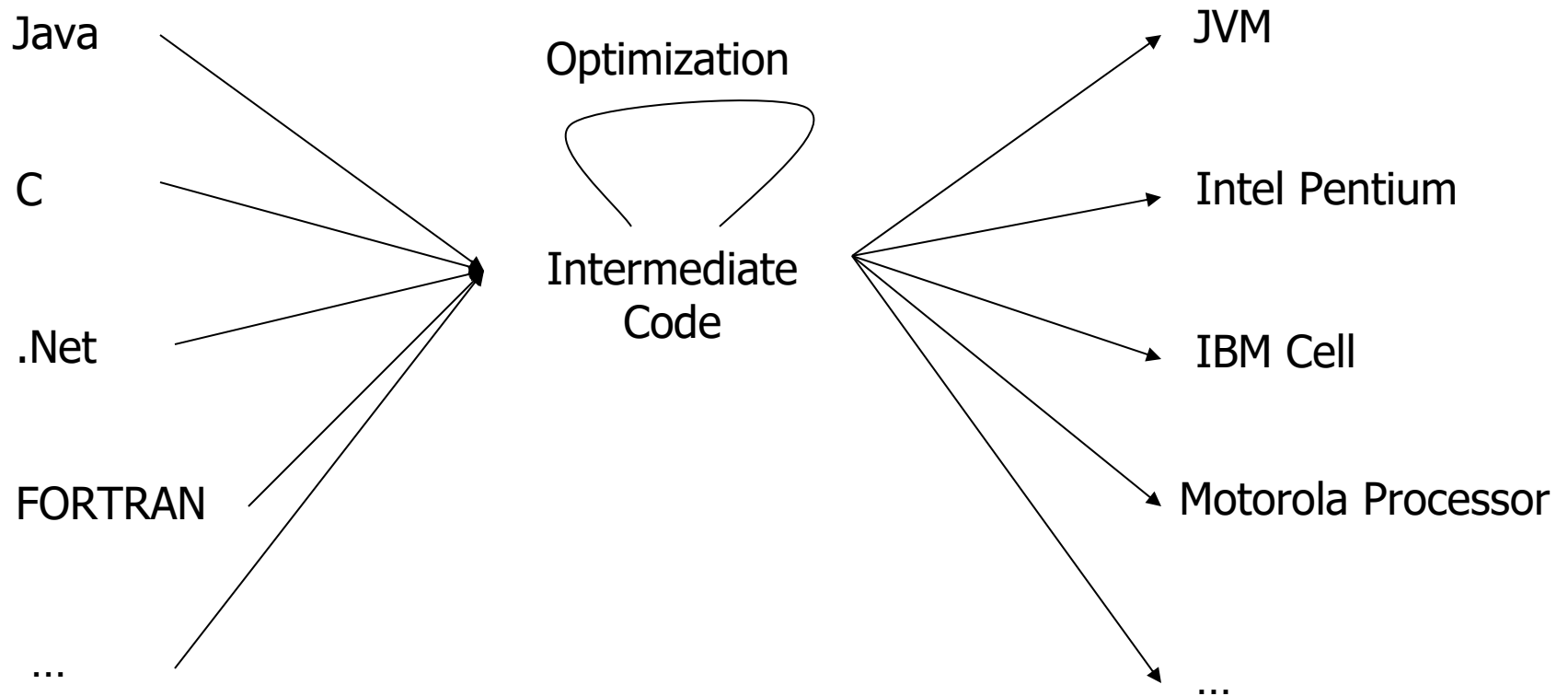
Compiler Modularity



Many Compilers



Many Compilers



Why Study Compilers? (1)

❑ Become a better programmer(!)

- ❑ Insight into interaction between languages, compilers, and hardware
- ❑ Understanding of implementation techniques
- ❑ What is all that stuff in the debugger anyway?
- ❑ Better intuition about what your code does

Why Study Compilers? (2)

- ❑ **Compiler techniques are everywhere**
 - ❑ Parsing (little languages, interpreters, XML)
 - ❑ Database engines, query languages
 - ❑ AI: domain-specific languages
 - ❑ Text processing
 - ❑ Tex/LaTeX -> dvi -> Postscript -> pdf
 - ❑ Hardware: VHDL; model-checking tools
 - ❑ Mathematics (Mathematica, Matlab)

Why Study Compilers? (3)

- ❑ Fascinating blend of theory and engineering
 - ❑ Direct applications of theory to practice
 - ❑ Parsing, scanning, static analysis
 - ❑ Some very difficult problems (NP-hard or worse)
 - ❑ Resource allocation, “optimization”, etc.
 - ❑ Need to come up with good-enough approximations/heuristics

Why Study Compilers? (4)

☐ Ideas from many parts of CSE

- ☐ AI: Greedy algorithms, heuristic search
- ☐ Algorithms: graph algorithms, dynamic programming, approximation algorithms
- ☐ Theory: Grammars, DFAs and PDAs, pattern matching, fixed-point algorithms
- ☐ Systems: Allocation & naming, synchronization, locality
- ☐ Architecture: pipelines, instruction set use, memory hierarchy management

Why Study Compilers? (5)

- ❑ You might even write a compiler some day!
 - ❑ You'll almost certainly write parsers and interpreters in some context if you haven't already

Some History (1)

❑ 1950's. Existence proof

- ❑ FORTRAN I (1954) – competitive with hand-optimized code

❑ 1960's

- ❑ New languages: ALGOL, LISP, COBOL, SIMULA
- ❑ Formal notations for syntax, esp. BNF
- ❑ Fundamental implementation techniques
 - ❑ Stack frames, recursive procedures, etc.

Some History (2)

❑ 1970's

- ❑ Syntax: formal methods for producing compiler front-ends; many theorems

❑ Late 1970's, 1980's

- ❑ New languages (functional; Smalltalk & object-oriented)
- ❑ New architectures (RISC machines, parallel machines, memory hierarchy issues)
- ❑ More attention to back-end issues

Some History (3)

❑ 1990s and beyond

- ❑ Compilation techniques appearing in many new places
 - ❑ Software analysis, verification, security
- ❑ Phased compilation – blurring the lines between “compile time” and “runtime”
 - ❑ Just-in-time compilers (JITs)
- ❑ Using machine learning techniques to control optimizations(!)
- ❑ Compiler technology critical to effective use of new hardware (RISC, Itanium, complex memory hierarchies)

Summary

- ❑ Compilers Translate Source Language to Target Language
- ❑ Compilers have several steps
 - ❑ Scanner
 - ❑ Parser
 - ❑ Semantic Analyzer
 - ❑ Intermediate Code Generator
 - ❑ Optimizer
 - ❑ Code Generator
- ❑ Symbol Table Used To Keep Track of Names Used in Program
- ❑ Front End and Back End Simplify Compiler Design
 - ❑ Introduction of new languages
 - ❑ Introduction of new hardware